CSE 344 – System Programming

Homework #5

Priority-Based Cargo Delivery Simulation

Name : FATİH EMRE OĞAN
Student ID : 230104004090
Course : CSE 344 – System Programming
Assignment : Homework #5

Table of Contents

1. System Design

1.1 Overview

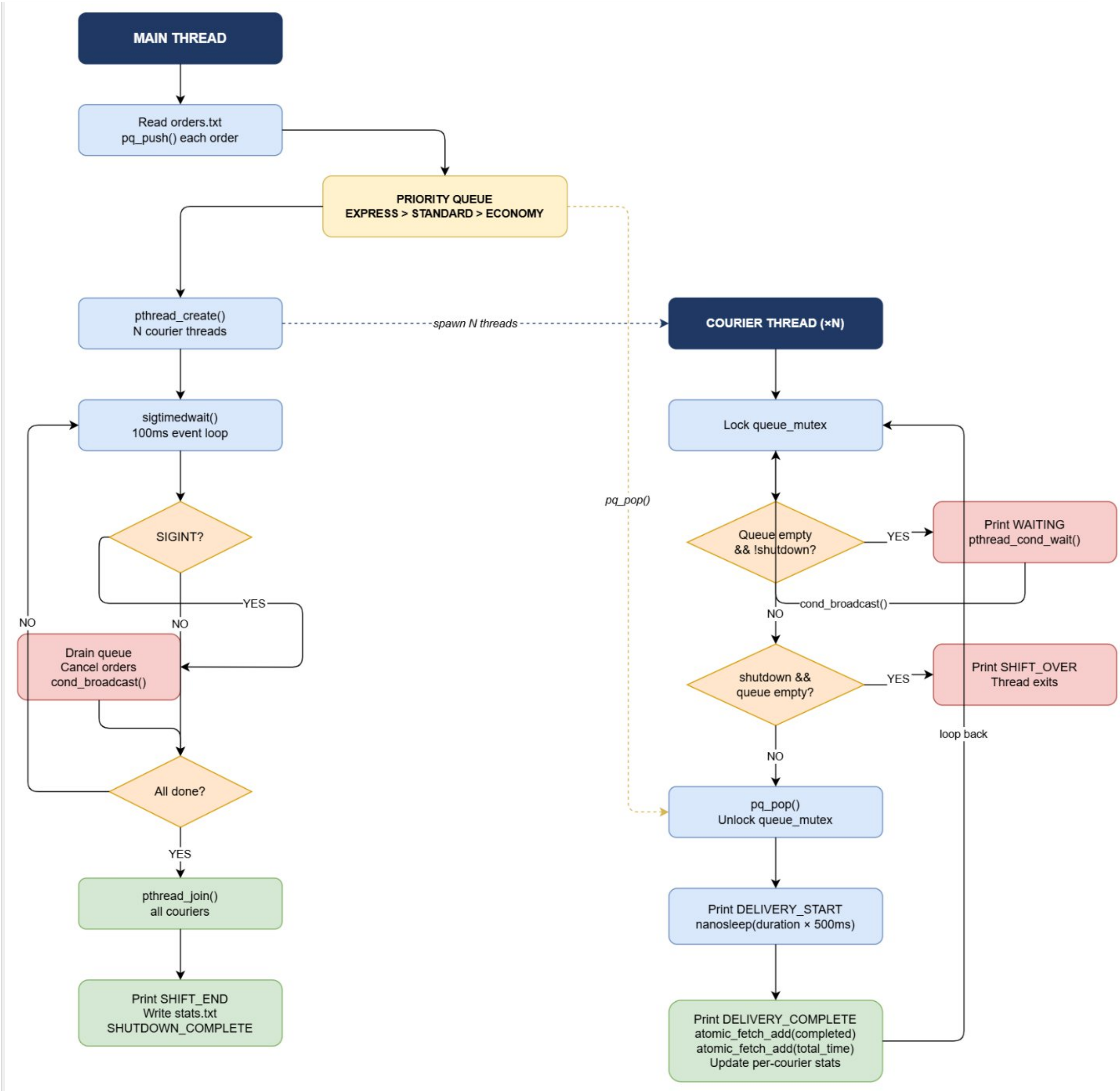
The cargoGTU program implements a thread-pool-based cargo delivery simulation. At startup, the main thread reads all delivery orders from an input file and inserts them into a shared priority queue. A fixed number of courier threads (specified by -n) are then created. These threads run concurrently, each picking up the highest-priority available order, simulating delivery by sleeping, then returning for the next order. The shift ends either when all orders are delivered or when a SIGINT signal is received.

1.2 Thread Pool Structure and Courier Lifecycle

The thread pool consists of exactly N courier threads created once at startup via pthread_create(). No new threads are created during the shift. Each courier thread executes the following lifecycle:

- Start: Thread is created and immediately enters its main loop.
- Wait: If the priority queue is empty and shutdown is not set, the courier prints [COURIER-N] WAITING and blocks on pthread_cond_wait(), releasing the queue mutex while sleeping.
- Wake-up: The courier is woken by pthread_cond_broadcast() (when a new order is available, or at shutdown).
- Exit check: If shutdown flag is set and queue is empty, the courier prints SHIFT_OVER and exits.
- Deliver: The courier pops the highest-priority order, releases the queue mutex, and simulates delivery by sleeping for (duration × 500 ms) using nanosleep().
- Update: Atomic counters (completed_orders, total_delivery_time) are updated. Per-courier stats are updated without a mutex since only the owning thread writes them.
- Repeat: The courier loops back to the Wait step.

1.3 Thread Pool Diagram



2. Priority Queue Design

2.1 Data Structure

The priority queue is implemented as a sorted singly-linked list (defined in queue.h / queue.c). Each node (QNode) holds one Order. The list is always kept in sorted order so that the head always points to the highest-priority order. Popping an order is therefore O(1); insertion is O(n) in the worst case.

2.2 Ordering Rules

Orders are sorted according to two criteria applied in order:

Priority Level	Numeric Value	Description
EXPRESS	1 (highest)	Time-critical, always processed first
STANDARD	2	Regular deliveries
ECONOMY	3 (lowest)	Non-urgent, processed last

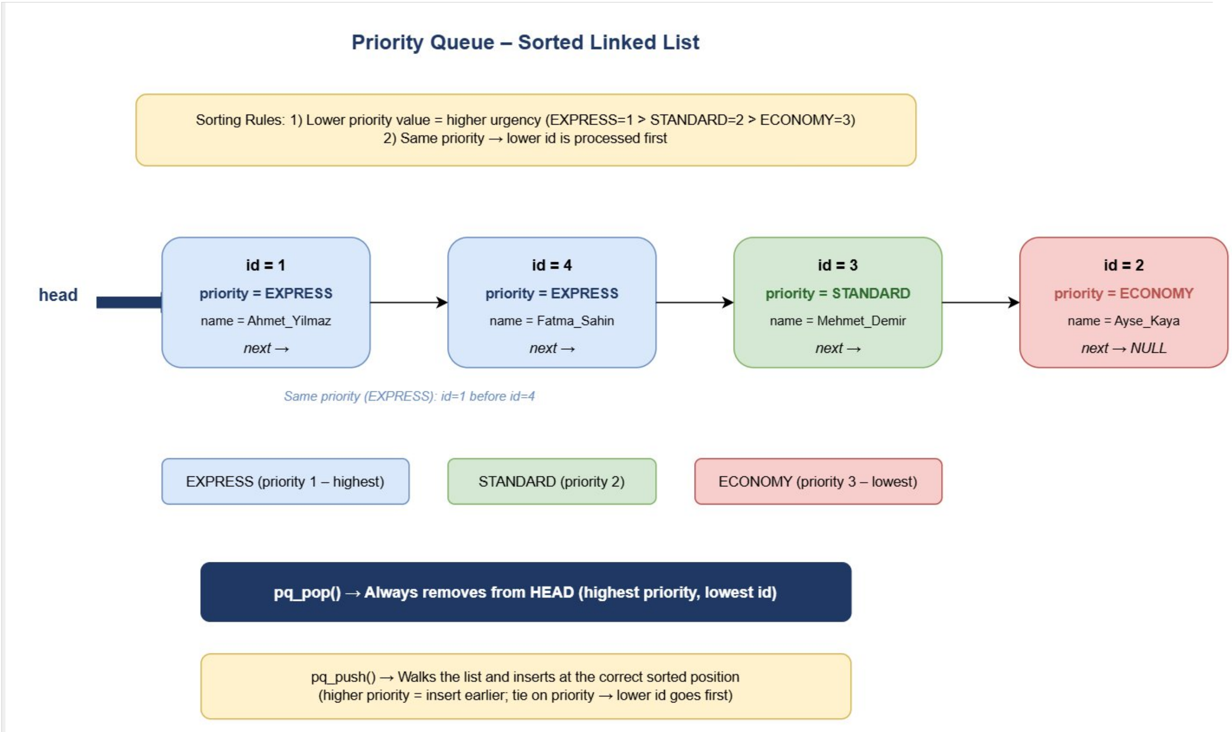
Tie-breaking rule: when two orders share the same priority, the one with the smaller id is processed first (FIFO within a priority level). This ensures deterministic, fair ordering.

2.3 pq_push() Algorithm

The insertion function walks the list from the head and stops at the first node where the new order has higher priority, or same priority with a lower id. The new node is then spliced in before that node. This keeps the list sorted at all times.

```
while (*cur):
    if new.priority < cur.priority  →  insert here
    if new.priority == cur.priority AND new.id < cur.id  →  insert here
    else: advance cur
```

2.4 Queue Diagram



3. Synchronization Strategy

Three synchronization primitives are used. Each protects a distinct resource to minimize lock contention.

Primitive	Protects	Details
queue_mutex (pthread_mutex_t)	g_queue (PriorityQueue)	Locked by any thread before pq_push, pq_pop, or pq_size. Also pairs with queue_cond for blocking.
queue_cond (pthread_cond_t)	Courier idle state	Couriers call pthread_cond_wait() when queue is empty. Main thread calls pthread_cond_broadcast() to wake all couriers on new work or shutdown.
log_mutex (pthread_mutex_t)	stdout (printf)	Held for the duration of every printf + fflush. Without it, output lines from different threads would interleave and become unreadable.

3.1 Atomic Counters

Three global statistics counters are declared with _Atomic (C11 stdatomic.h) and updated with atomic_fetch_add():

- g_completed_orders – incremented by a courier after each delivery.
- g_cancelled_orders – incremented by the main thread for each order drained from the queue on SIGINT.
- g_total_delivery_time – the courier adds (duration × 500 ms) after each delivery.

Using _Atomic guarantees that each read-modify-write is indivisible without requiring a mutex, which reduces contention at high courier counts. Using a mutex for these counters would be incorrect per the assignment specification and would incur a grading penalty.

3.2 Per-Courier Stats

Each CourierStats struct (completed, total_time_ms) is written exclusively by the courier thread that owns it. No mutex is needed because only one thread ever writes, and the main thread reads these fields only after pthread_join(), which provides the required memory-visibility guarantee.

3.3 No Busy Waiting

Couriers never spin-poll the queue. When the queue is empty, a courier calls pthread_cond_wait(), which atomically releases the queue mutex and puts the thread to sleep. The OS wakes the thread only when pthread_cond_broadcast() or pthread_cond_signal() is called. This is zero-CPU idle waiting, as required by the specification.

4. SIGINT Handling and Clean Shutdown

4.1 Signal Masking Strategy

SIGINT is blocked in all threads using pthread_sigmask(SIG_BLOCK, ...) in the main thread before any courier threads are spawned. Because child threads inherit the signal mask of their parent, SIGINT is also blocked in every courier thread. This means the signal is never delivered asynchronously to any thread.

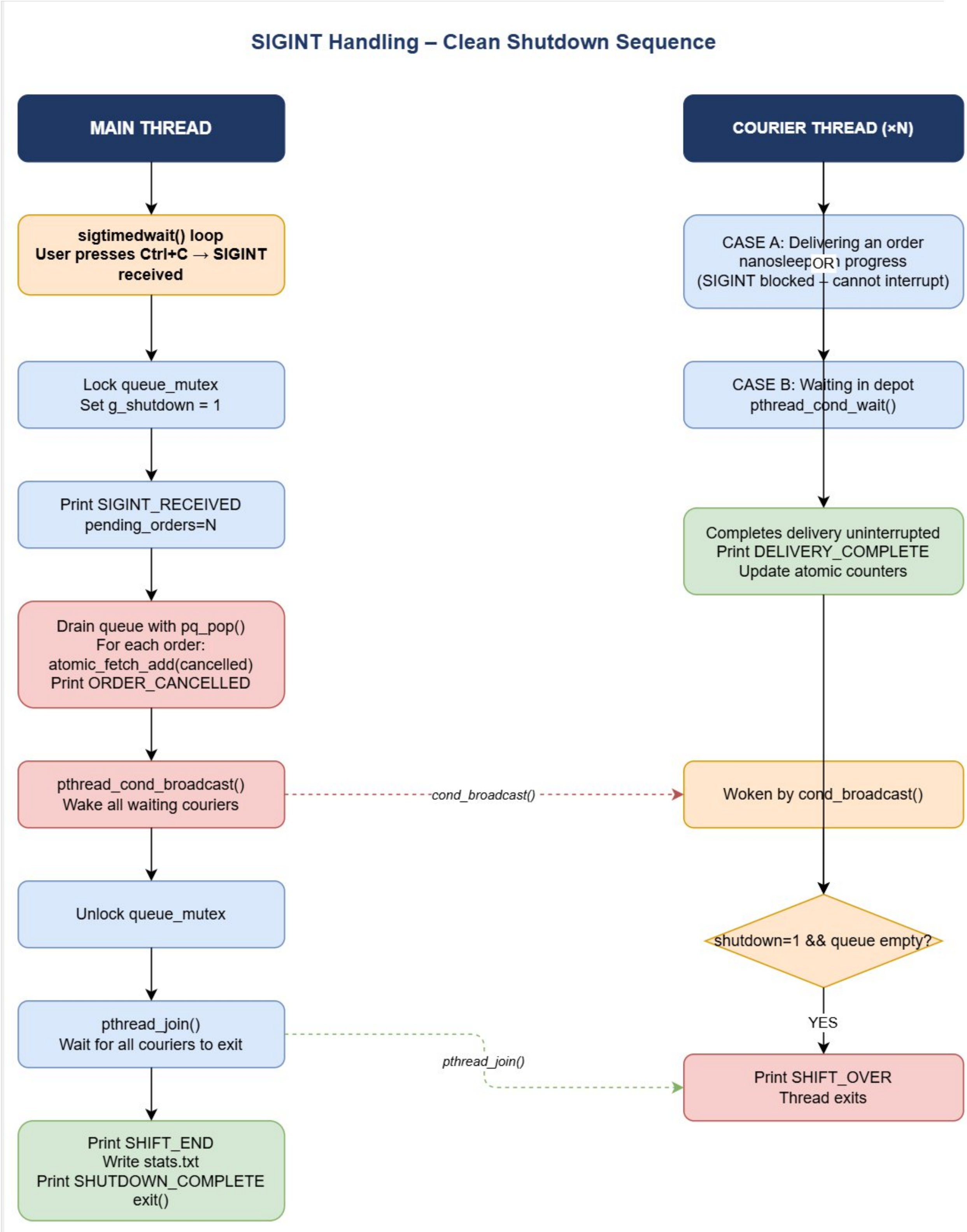
4.2 sigtimedwait() Event Loop

The main thread uses sigtimedwait() in a 100 ms polling loop to detect SIGINT synchronously. Between each poll it also checks if all orders are accounted for (completed + cancelled == total_orders), which is the natural-end condition.

4.3 Step-by-Step Shutdown Sequence

- Step 1 – SIGINT detected: sigtimedwait() returns SIGINT.
- Step 2 – Set shutdown flag: queue_mutex is locked, g_shutdown = 1 is written.
- Step 3 – Log SIGINT: [CARGOGTU] SIGINT_RECEIVED pending_orders=N is printed.
- Step 4 – Cancel pending orders: The queue is drained with pq_pop(). For each popped order, g_cancelled_orders is atomically incremented and [CARGOGTU] ORDER_CANCELLED is printed.
- Step 5 – Wake couriers: pthread_cond_broadcast() is called so all couriers blocked in pthread_cond_wait() wake up and see the shutdown flag.
- Step 6 – Release queue_mutex.
- Step 7 – Active deliveries complete: Couriers currently sleeping in nanosleep() finish uninterrupted (SIGINT is blocked in them, so nanosleep cannot be interrupted).
- Step 8 – Waiting couriers exit: Each woken courier checks g_shutdown and pq_size() == 0, prints SHIFT_OVER, and returns.
- Step 9 – pthread_join(): Main thread joins all couriers, blocking until all have exited.
- Step 10 – Final output: [CARGOGTU] SHIFT_END and [CARGOGTU] SHUTDOWN_COMPLETE are printed.
- Step 11 – Statistics file written: write_stats() writes the summary to the -s file.
- Step 12 – Resource cleanup: pq_destroy(), mutex/condvar destroy, free().

4.4 SIGINT Diagram



5. Test Scenarios

All five scenarios were run separately. Commands and verification points are listed below. Uncut console screenshots follow each command.

Test 1: Normal Completion – Mixed Orders

Field	Value
Scenario #	1
Couriers (-n)	3
Input file (-i)	10orders_mix.txt
Extra condition	None – normal run

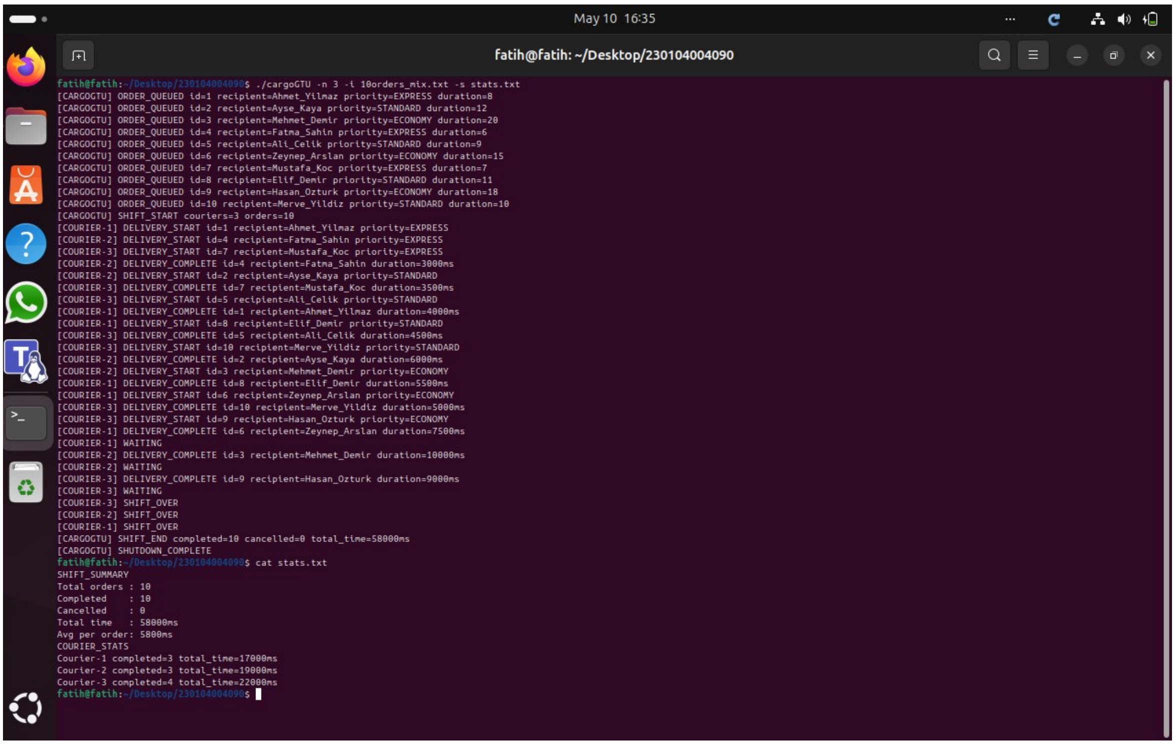
Command

```
make
./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
```

What to Verify

- All 10 orders are completed (completed=10, cancelled=0 in SHIFT_END).
- Priority ordering is respected: all EXPRESS orders appear before STANDARD, STANDARD before ECONOMY in DELIVERY_START lines.
- stats.txt is created and contains correct SHIFT_SUMMARY and COURIER_STATS sections.
- No duplicate DELIVERY_START id values appear in the output.

Console Output Screenshot



Test 2: Same-Priority FIFO – Economy Only

Field	Value
Scenario #	2
Couriers (-n)	10
Input file (-i)	10orders_economy.txt
Extra condition	None – all orders are ECONOMY priority

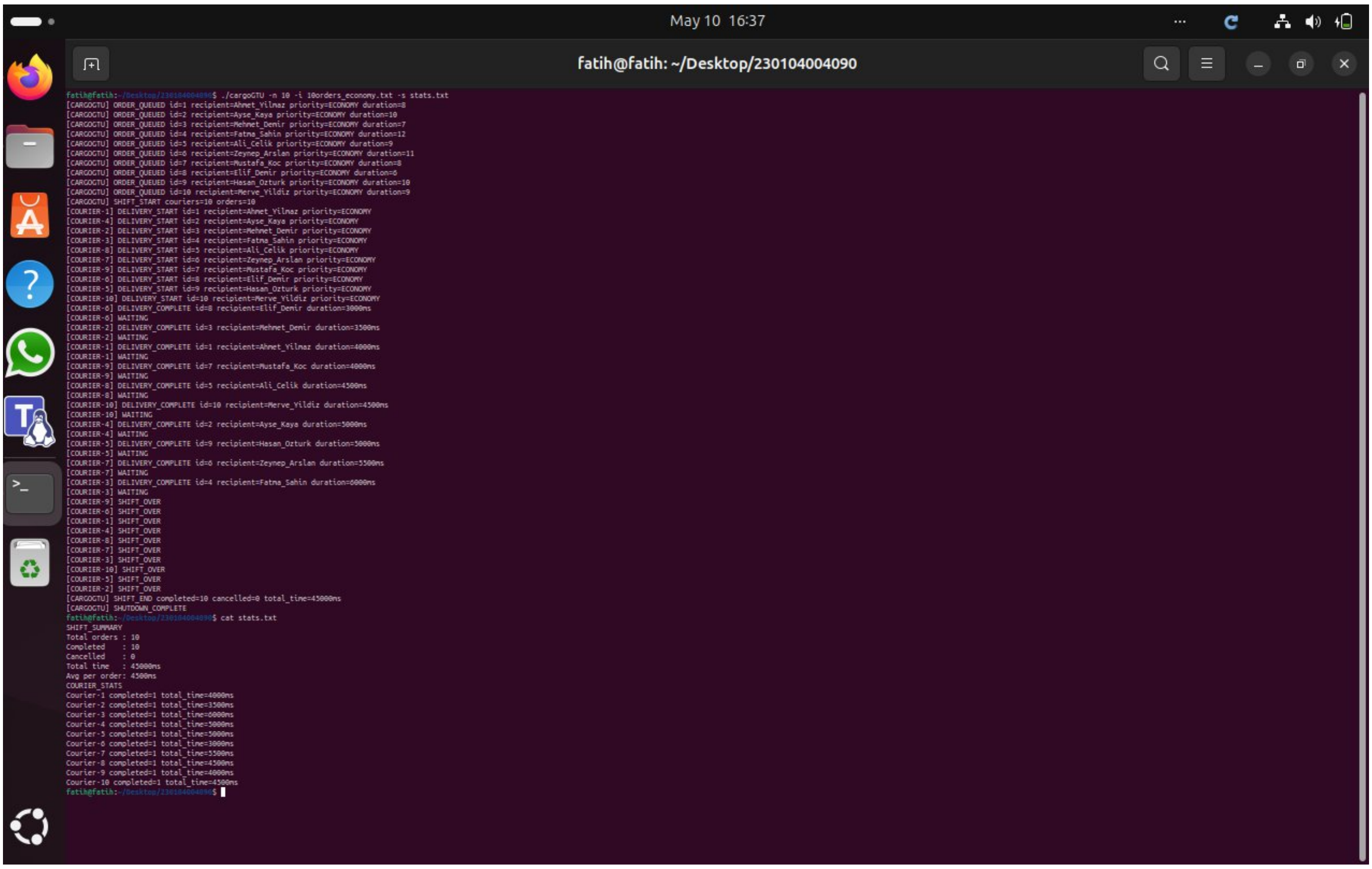
Command

```
make
./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
```

What to Verify

- All 10 orders are completed without duplication.
- Since all orders share ECONOMY priority, they must be taken in ascending ID order (id=1 before id=2, etc.).
- Multiple couriers participate concurrently because 10 threads are available and 10 orders exist.
- No courier processes the same order twice.

Console Output Screenshot



Test 3: SIGINT Mid-Shift – Partial Cancellation

Field	Value
Scenario #	3
Couriers (-n)	2
Input file (-i)	20orders_mix.txt
Extra condition	Send SIGINT (Ctrl+C) approximately 2 seconds after launch

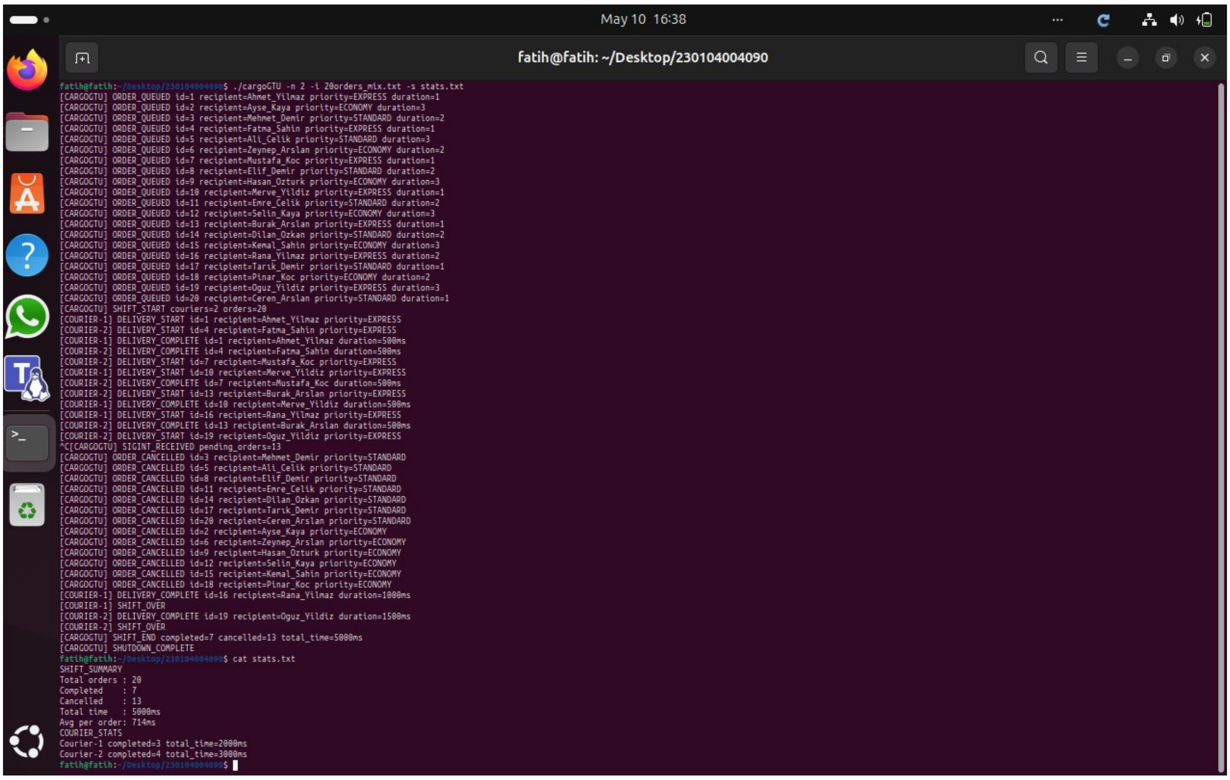
Command

```
make
./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt
```

What to Verify

- Active deliveries complete without interruption (DELIVERY_COMPLETE appears for in-progress orders).
- Pending orders in the queue are cancelled (ORDER_CANCELLED lines appear).
- SHIFT_END shows completed + cancelled == 20 (all orders accounted for).
- Program exits cleanly; running 'ps aux | grep cargoGTU' after exit shows no zombie process.
- stats.txt is written correctly even after SIGINT.

Console Output Screenshot



Test 4: Memory Leak Check – Valgrind

Field	Value
Scenario #	4
Couriers (-n)	3
Input file (-i)	10orders_mix.txt
Extra condition	Run under Valgrind with full leak check

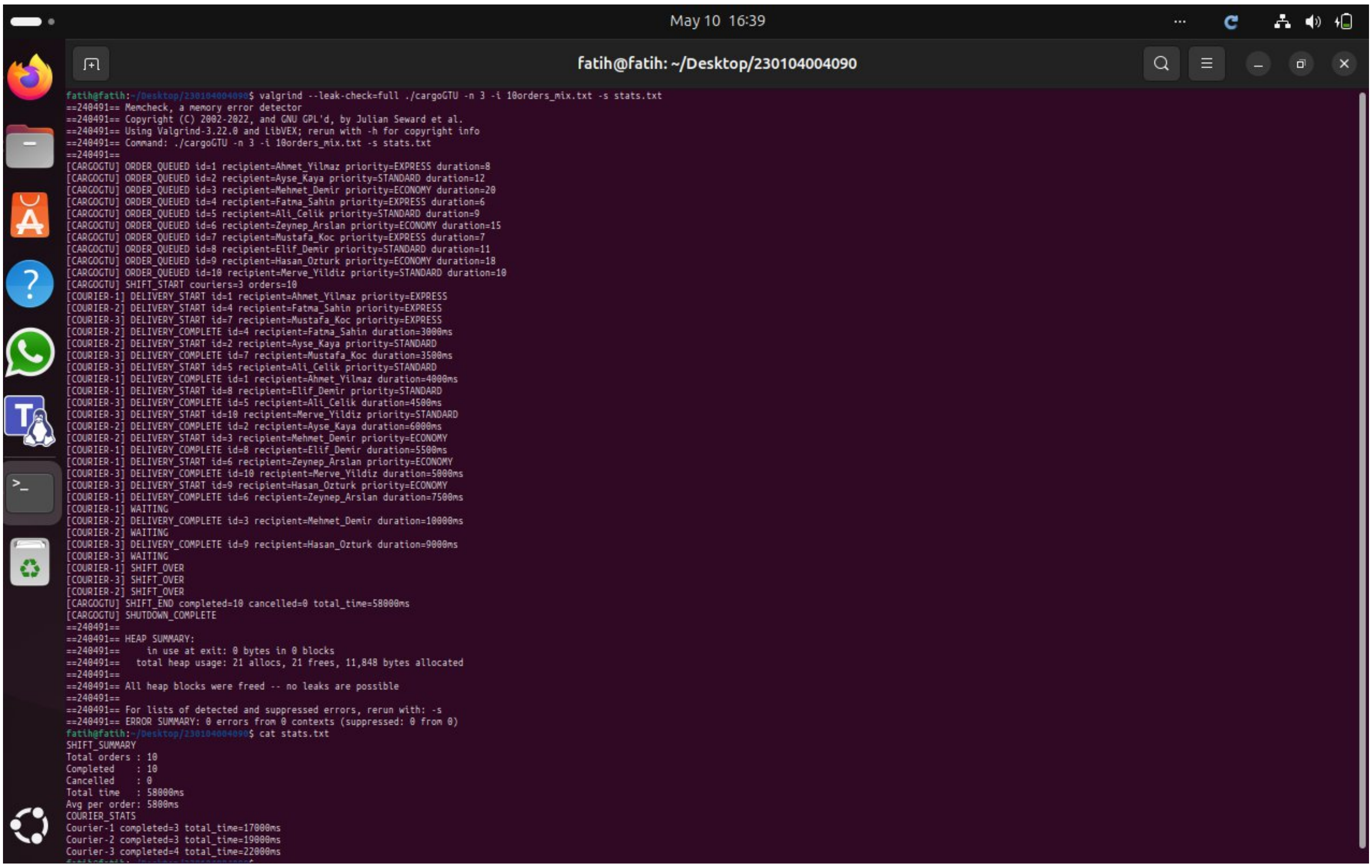
Command

```
make
valgrind --leak-check=full --error-exitcode=1 \
  ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
```

What to Verify

- Valgrind reports 0 errors and 0 memory leaks (definitely lost: 0 bytes).
- All heap memory allocated for stats, args, tids, and queue nodes is freed before exit.
- All pthread_mutex and pthread_cond objects are destroyed before exit.
- Full uncut Valgrind output is shown in the screenshot.

Console Output Screenshot



Test 5: High Concurrency – Priority Consistency

Field	Value
Scenario #	5
Couriers (-n)	10
Input file (-i)	20orders_mix.txt
Extra condition	Run the program 3 or more times consecutively

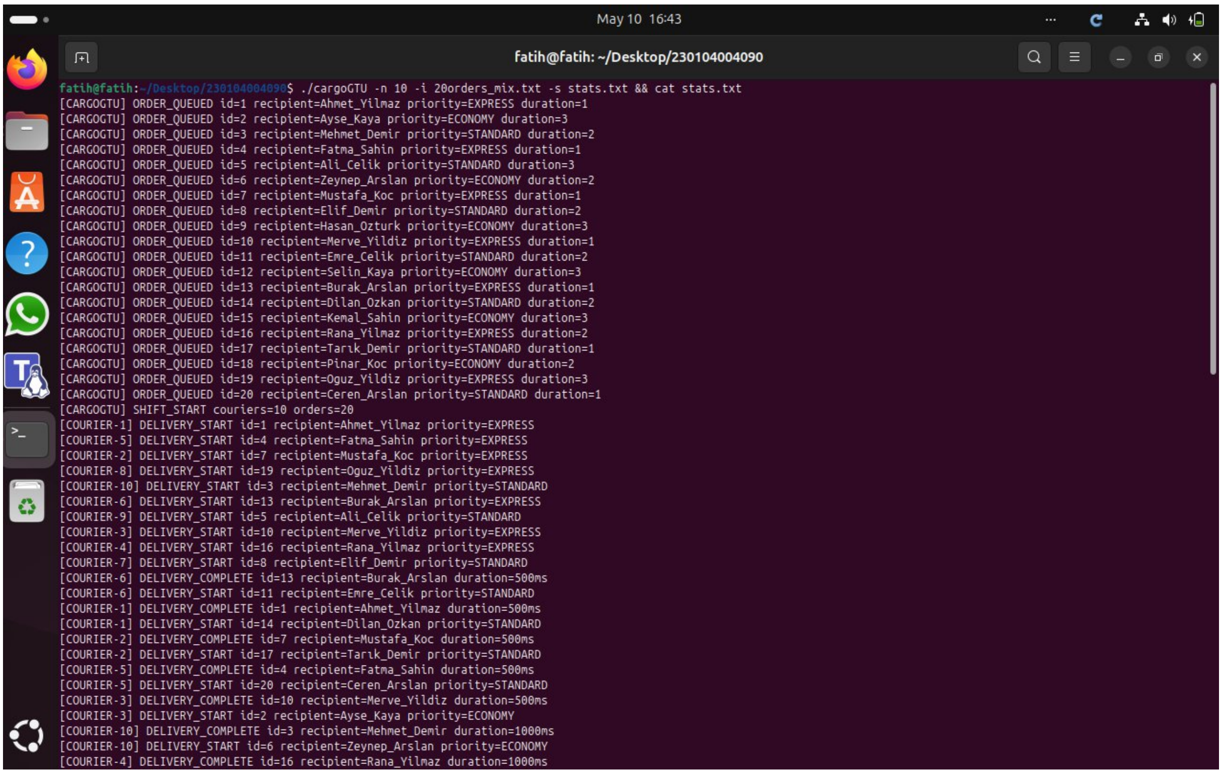
Command

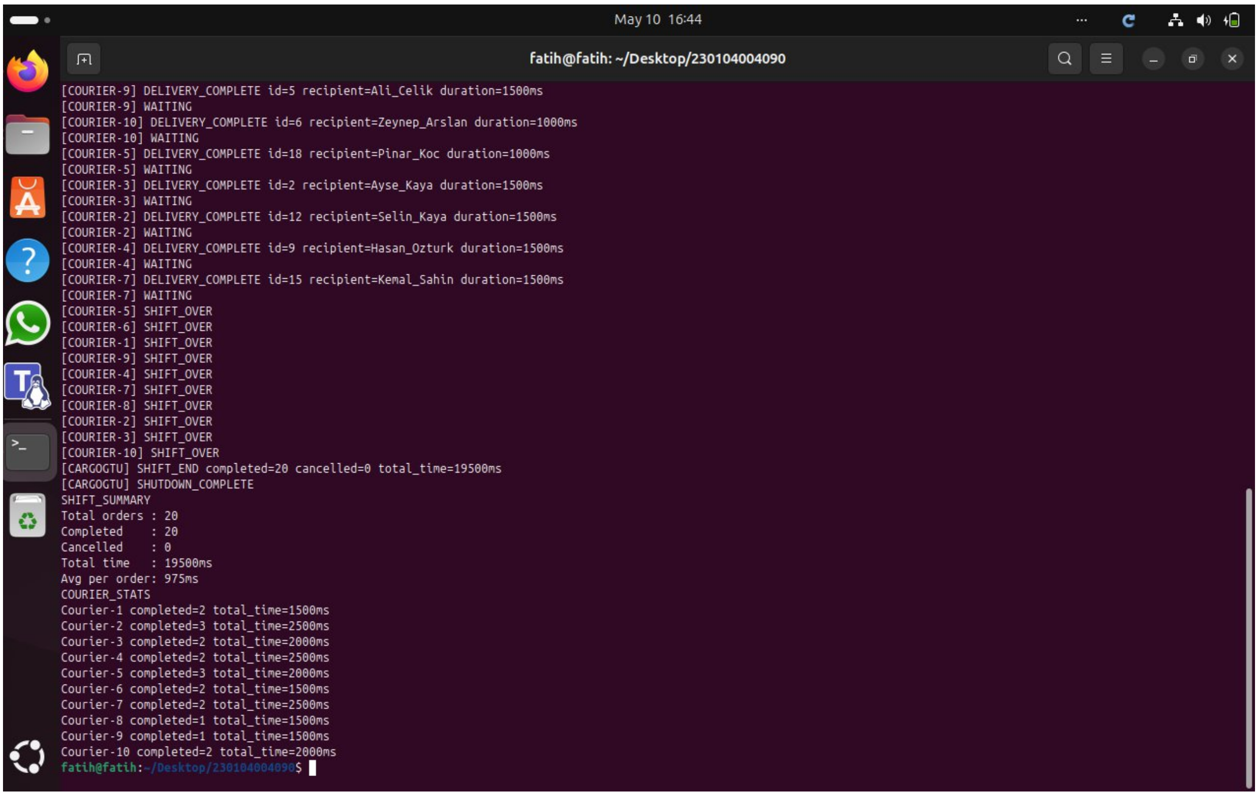
```
make
./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
# Run again:
./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
# Run again:
./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
```

What to Verify

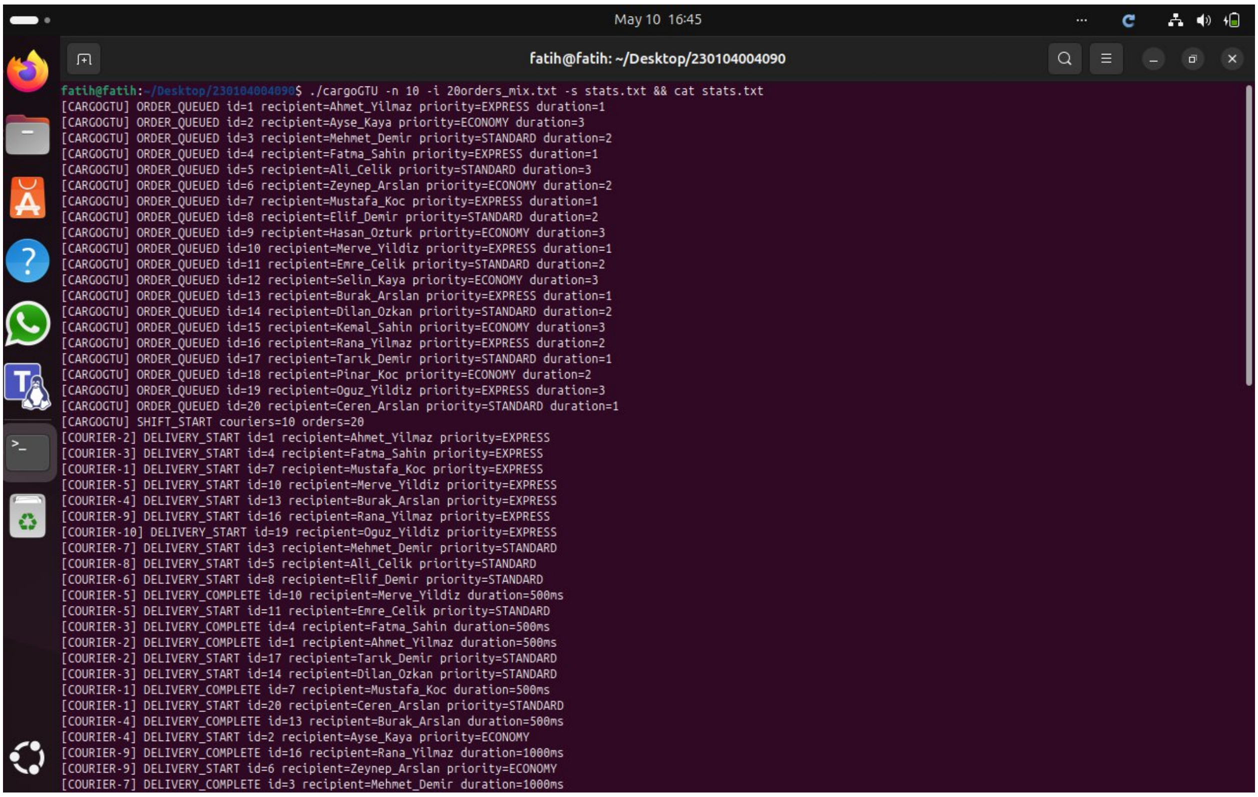
- SHIFT_END completed count is identical across all runs (always 20).
- No duplicate DELIVERY_START id appears in any single run's output.
- Priority order is maintained even under high concurrency (EXPRESS always before STANDARD, STANDARD before ECONOMY).
- No race conditions or deadlocks observed across multiple runs.

Console Output Screenshots

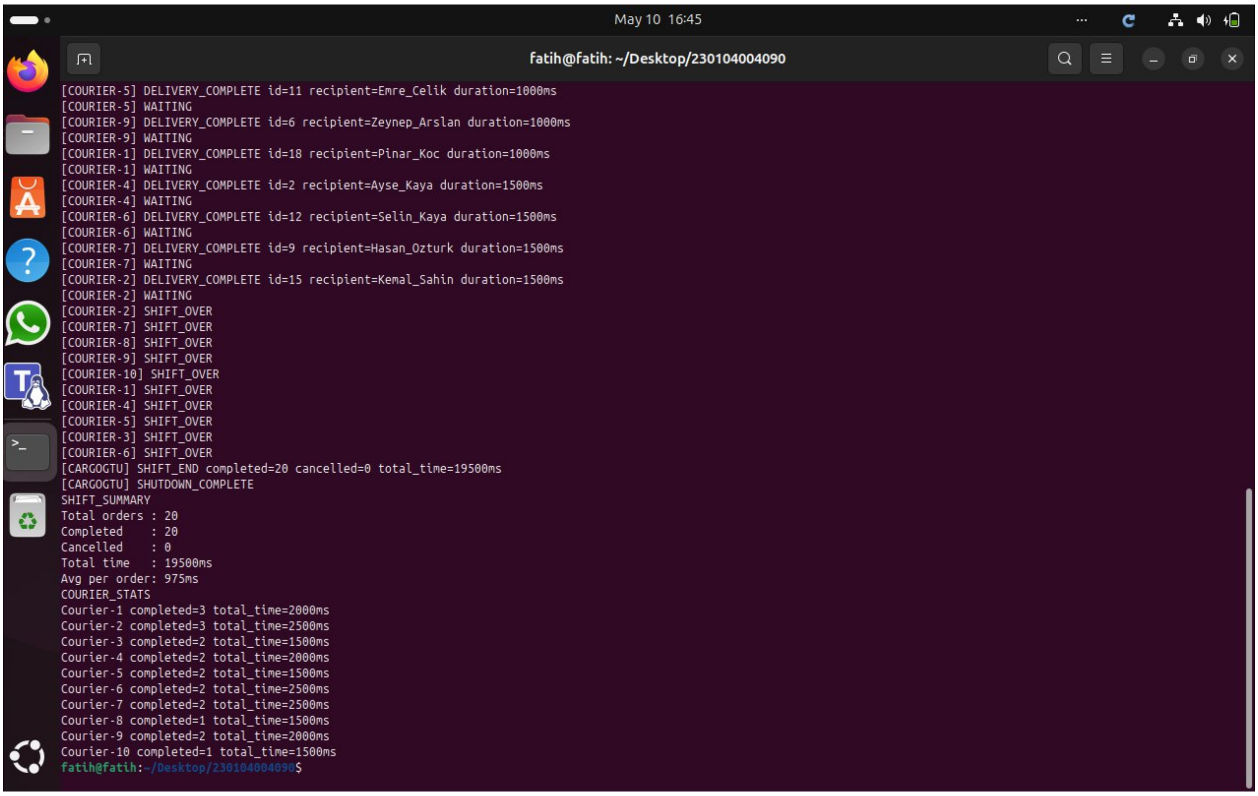




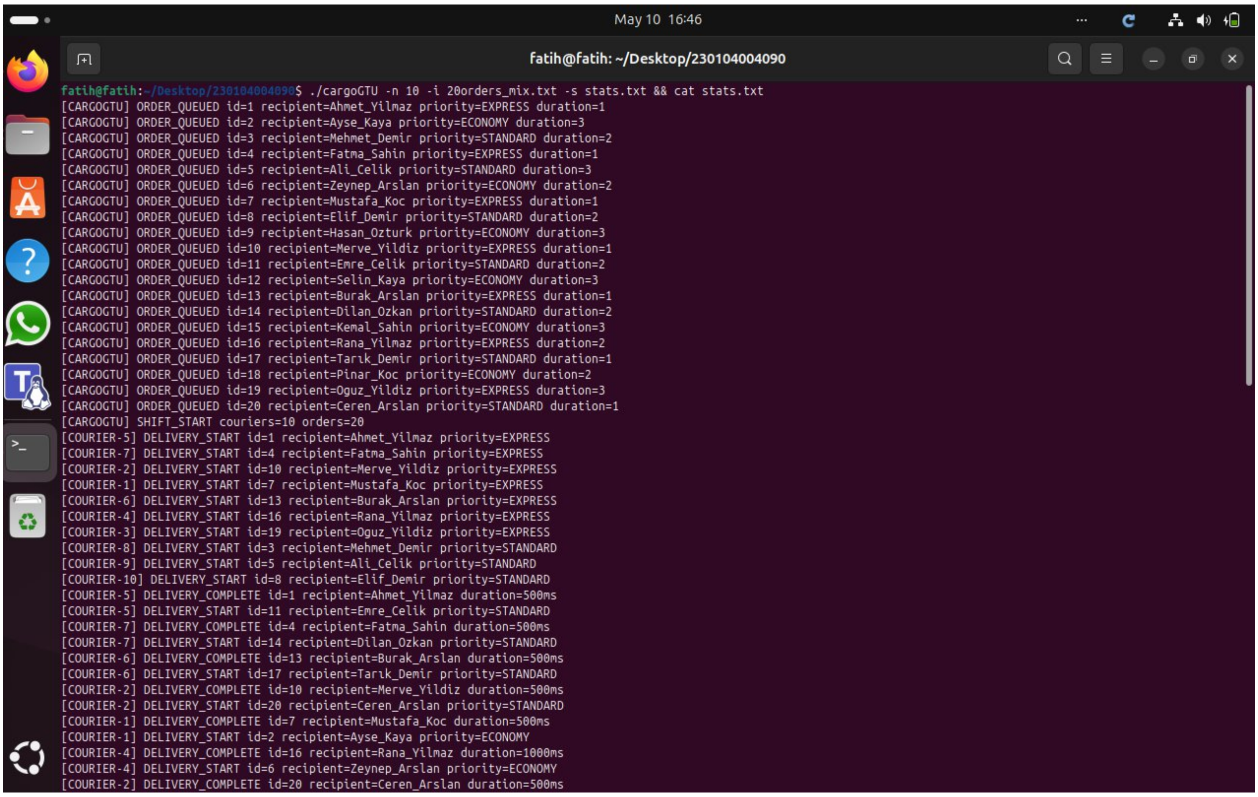

```
[COURIER-9] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=1500ms
[COURIER-9] WAITING
[COURIER-10] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-10] WAITING
[COURIER-5] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-5] WAITING
[COURIER-3] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=1500ms
[COURIER-3] WAITING
[COURIER-2] DELIVERY_COMPLETE id=12 recipient=Selin_Kaya duration=1500ms
[COURIER-2] WAITING
[COURIER-4] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
[COURIER-4] WAITING
[COURIER-7] DELIVERY_COMPLETE id=15 recipient=Kemal_Sahin duration=1500ms
[COURIER-7] WAITING
[COURIER-5] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHUTDOWN_COMPLETE
SHIFT_SUMMARY
Total orders : 20
Completed    : 20
Cancelled    : 0
Total time   : 19500ms
Avg per order: 975ms
COURIER_STATS
Courier-1 completed=2 total_time=1500ms
Courier-2 completed=3 total_time=2500ms
Courier-3 completed=2 total_time=2000ms
Courier-4 completed=2 total_time=2500ms
Courier-5 completed=3 total_time=2000ms
Courier-6 completed=2 total_time=1500ms
Courier-7 completed=2 total_time=2500ms
Courier-8 completed=1 total_time=1500ms
Courier-9 completed=1 total_time=1500ms
Courier-10 completed=2 total_time=2000ms
fatih@fatih:~/Desktop/230104004090$
```



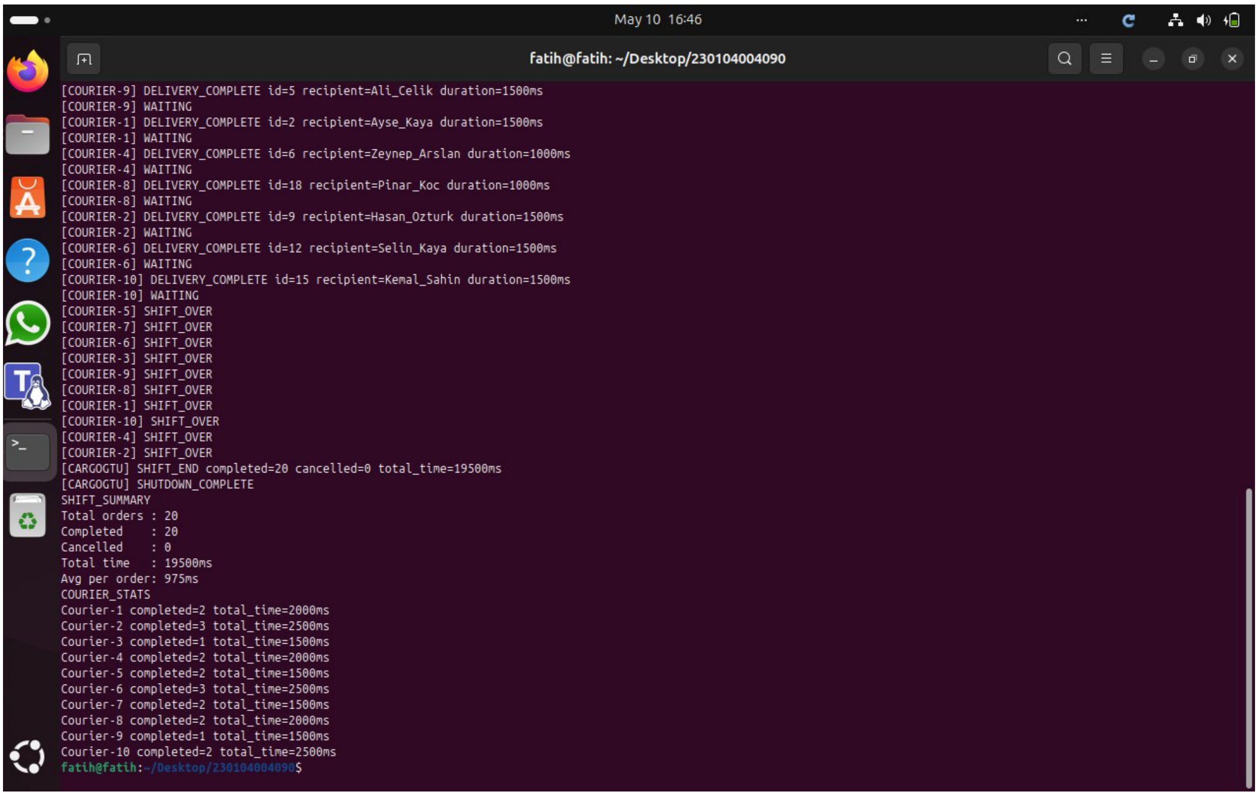

```
fatih@fatih:~/Desktop/230104004090$ ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt && cat stats.txt
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarık_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[CARGOGTU] SHIFT_START couriers=10 orders=20
[COURIER-2] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-3] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-5] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-4] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-9] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-10] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[COURIER-7] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-8] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-6] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-5] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-5] DELIVERY_START id=11 recipient=Emre_Celik priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-2] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-2] DELIVERY_START id=17 recipient=Tarık_Demir priority=STANDARD
[COURIER-3] DELIVERY_START id=14 recipient=Dilan_Ozkan priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-1] DELIVERY_START id=20 recipient=Ceren_Arslan priority=STANDARD
[COURIER-4] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-4] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-9] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-9] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-7] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=1000ms
```




```
[COURIER-5] DELIVERY_COMPLETE id=11 recipient=Emre_Celik duration=1000ms
[COURIER-5] WAITING
[COURIER-9] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-9] WAITING
[COURIER-1] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-1] WAITING
[COURIER-4] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=1500ms
[COURIER-4] WAITING
[COURIER-6] DELIVERY_COMPLETE id=12 recipient=Selin_Kaya duration=1500ms
[COURIER-6] WAITING
[COURIER-7] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
[COURIER-7] WAITING
[COURIER-2] DELIVERY_COMPLETE id=15 recipient=Kemal_Sahin duration=1500ms
[COURIER-2] WAITING
[COURIER-2] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHUTDOWN_COMPLETE
SHIFT_SUMMARY
Total orders : 20
Completed    : 20
Cancelled    : 0
Total time   : 19500ms
Avg per order: 975ms
COURIER_STATS
Courier-1 completed=3 total_time=2000ms
Courier-2 completed=3 total_time=2500ms
Courier-3 completed=2 total_time=1500ms
Courier-4 completed=2 total_time=2000ms
Courier-5 completed=2 total_time=1500ms
Courier-6 completed=2 total_time=2500ms
Courier-7 completed=2 total_time=2500ms
Courier-8 completed=1 total_time=1500ms
Courier-9 completed=2 total_time=2000ms
Courier-10 completed=1 total_time=1500ms
fatih@fatih:~/Desktop/230104004090$
```




```
fatih@fatih:~/Desktop/230104004090$ ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt && cat stats.txt
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[CARGOGTU] SHIFT_START couriers=10 orders=20
[COURIER-5] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-7] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-2] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-6] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-4] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-3] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[COURIER-8] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-9] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-10] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-5] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-5] DELIVERY_START id=11 recipient=Emre_Celik priority=STANDARD
[COURIER-7] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-7] DELIVERY_START id=14 recipient=Dilan_Ozkan priority=STANDARD
[COURIER-6] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-6] DELIVERY_START id=17 recipient=Tarik_Demir priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-2] DELIVERY_START id=20 recipient=Ceren_Arslan priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-1] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-4] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-4] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=20 recipient=Ceren_Arslan duration=500ms
```


May 10 16:46
fatih@fatih: ~/Desktop/230104004090
[COURIER-9] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=1500ms
[COURIER-9] WAITING
[COURIER-1] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=1500ms
[COURIER-1] WAITING
[COURIER-4] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-4] WAITING
[COURIER-8] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-8] WAITING
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
[COURIER-2] WAITING
[COURIER-6] DELIVERY_COMPLETE id=12 recipient=Selin_Kaya duration=1500ms
[COURIER-6] WAITING
[COURIER-10] DELIVERY_COMPLETE id=15 recipient=Kemal_Sahin duration=1500ms
[COURIER-10] WAITING
[COURIER-5] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHUTDOWN_COMPLETE
SHIFT_SUMMARY
Total orders : 20
Completed   : 20
Cancelled   : 0
Total time   : 19500ms
Avg per order: 975ms
COURIER_STATS
Courier-1 completed=2 total_time=2000ms
Courier-2 completed=3 total_time=2500ms
Courier-3 completed=1 total_time=1500ms
Courier-4 completed=2 total_time=2000ms
Courier-5 completed=2 total_time=1500ms
Courier-6 completed=3 total_time=2500ms
Courier-7 completed=2 total_time=1500ms
Courier-8 completed=2 total_time=2000ms
Courier-9 completed=1 total_time=1500ms
Courier-10 completed=2 total_time=2500ms
fatih@fatih:~/Desktop/230104004090$

6. Challenges & Solutions

Challenge 1 – Spurious Wakeups from pthread_cond_wait()

Problem: pthread_cond_wait() can return even when the condition it is waiting for (queue non-empty or shutdown set) is not yet true. This is called a spurious wakeup and is allowed by the POSIX standard. Without a guard loop, a courier would wake up, see an empty queue, and crash when calling pq_pop() on nothing, or silently exit prematurely.

Solution: The wait is wrapped in a while loop that re-checks the condition on every wakeup:

```
while (pq_size(queue) == 0 && !(*shutdown)) {
    pthread_cond_wait(queue_cond, queue_mutex);
}
```

This ensures the courier only proceeds when the queue is actually non-empty or shutdown has been set, regardless of whether the wakeup was spurious or genuine.

Challenge 2 – Safe SIGINT Handling in a Multi-Threaded Program

Problem: Handling SIGINT with a traditional signal handler in a multi-threaded program is unsafe because most C library functions (including printf and malloc) are not async-signal-safe. If a courier thread is inside printf when the signal fires, calling printf again from the signal handler causes undefined behavior.

Solution: SIGINT is blocked in all threads using pthread_sigmask(SIG_BLOCK) in the main thread before spawning any couriers (child threads inherit the mask). The main thread then catches the signal synchronously using sigtimedwait() in a polling loop. This approach is completely safe because no signal handler is used; all shutdown logic runs in normal program flow. The 100 ms timeout also allows the loop to check for natural completion between SIGINT checks.

Challenge 3 – Preventing the WAITING Message From Printing Multiple Times

Problem: When a courier wakes due to a spurious wakeup and the queue is still empty, it would re-enter the cond_wait loop and potentially print WAITING again, producing duplicate log lines.

Solution: The WAITING message is printed only once, in the if block that runs before the while loop. The while loop itself only calls pthread_cond_wait() and never prints. This guarantees WAITING is printed exactly once per idle period.